

01 / Project snapshot

IOP - Native development field guide

Prepared for offline use on 2026-10-09. Business-code snapshot: develop at 9fb1da9. IOP-199 is integrated, published to origin and activated in the local Docker stack. This guide is maintained under IOP-200; it does not declare a whole-platform release.

Your selected setup: React/Vite and the NestJS API run on the Mac. PostgreSQL stays in its existing Docker container, using the same database, volume, accounts and data. Native means the two application processes run outside Docker; Docker Desktop is still required for the database.

Delivered area	Current behavior
Data Analysis / OIP	CSV admission, original-file review, preparation, historical reports, trends, comparisons, KPIs and source-component Pareto.
Access	Local password login, four profiles, scoped permissions, profile administration and retained identities.
Shift Handover	Meeting preparation, personal Journal, matrix, durable issues, revisions, notices and IOP-199 category workflows.
Workforce	Teams, schedules, imports, weekly planning, dated assignments, floating duties and history.
Maintenance	Work board, ownership, priorities, status, outcomes and reviewed include/exclude repair scope.
Digital Asset Record	Stable registry, exact aliases, validation, retirement and source-authorized history.

Recent changes: opening details acknowledges notifications; entry location uses the actual dated assignment; Safety omits equipment; a single Technical issue choice classifies Problems/Performance; Success closes explicit references; Team Leader Information has images/display dates/Start visibility; People reuses Workforce.

Read this guide in order: architecture and files (02-04), preparation (05-06), daily startup (07), following a change (08), checks and schema work (09), troubleshooting (10), remaining scope (11), and offline/review routine (12).

Scope of verification: IOP-199 tests passed. The native startup, compiler watch, API restart and Vite proxy recipe was exercised against the existing database. Bootstrap installation commands are preparation instructions, not a claim that a fresh laptop was provisioned. Health reports process liveness only.

Editable source: docs/development/native-development-guide.md. Renderer: docs/development/pdf/build_native_guide.py. This is a checked runbook, not an automatic setup/reset script.

02 / Architecture and repository layout

IOP is a modular monolith: one API process composes modules with distinct ownership. There are no internal HTTP calls between these modules and no deployed worker service. The separate web and database processes do not turn each business module into a service.

```
Browser: React + Vite      http://127.0.0.1:5173
| same-origin /api and /health proxy
API: NestJS + module use cases 127.0.0.1:3000
| pg Pool -> pinned authorized transaction
PostgreSQL in Docker      127.0.0.1:5442
| existing iop_local database / persistent volume
```

Location	Read or change it for
ARCHITECTURE.md	Accepted implementation boundaries and composition overview.
apps/api/src/main.ts	API bootstrap, shutdown and listener.
apps/api/src/host/	Controllers, DTOs, runtime composition, configuration and HTTP errors.
apps/api/src/modules/	Owning domain rules, application use cases and adapters.
apps/api/src/persistence/site-operation.ts	Scoped authorization, one connection, transaction and RLS context.
apps/web/src/main.tsx	React root and shared base styles.
apps/web/src/host/	Composition, profile-aware shell, navigation and Start.
apps/web/src/features/	Browser domain/application contracts, HTTP gateways and React views.
apps/web/src/design/	Shared controls, identity tokens, surfaces and typography.
infra/database/	Role setup, ordered SQL migrations, CLI and PostgreSQL tests.
scripts/local/	Docker orchestration and explicit operator/demo tools.
config/ and .local-platform/	Tracked examples/source presets versus private live installation settings.
docs/planning/	Backlog, permanent items, active plans and completed evidence.

Domain code owns values and invariants. Application code owns use cases and ports. Adapters handle React, HTTP, SQL and source formats. Host composition connects them. Keep customer labels and source schemas in configuration/integration adapters.

Build output in dist/ is disposable. Database state is in the Docker volume. Empty/future package or host placeholders are not delivered features.

03 / Backend module map

All paths in this table are below `apps/api/src/modules/`. M numbers describe product areas; directories describe actual code ownership.

Module directory	Responsibility and first files
<code>platform-core/</code>	Scope and configured locations. <code>adapters/configuration/locations.ts</code> and <code>adapters/postgres/site-ownership.ts</code> validate location structure and site ownership.
<code>users-rbac/</code>	Current permission decisions, four-profile bundles and administration. <code>domain/profiles.ts</code> , <code>application/authorize-site.ts</code> , <code>application/administration.ts</code> ; <code>adapters/postgres/</code> holds directory and lookup SQL.
<code>authentication/</code>	Password verification, throttling, sessions and revocation. <code>application/authentication.ts</code> ; <code>adapters/node-crypto.ts</code> and <code>adapters/postgres.ts</code> .
<code>integrations/</code>	CSV/schedule decoding, source mappings, RAW/import receipt and atomic admission. <code>application/import-workflow.ts</code> ; <code>adapters/csv/</code> , <code>adapters/schedule/</code> and <code>adapters/postgres/import-batches.ts</code> .
<code>oip/</code>	Data Analysis policy and reporting. <code>application/reporting.ts</code> , <code>application/data-explorer.ts</code> and <code>application/ports.ts</code> ; <code>domain/</code> models and <code>adapters/postgres/</code> reports/projections.
<code>shift-handover/</code>	Entry validation, issue state, revisions, configured categories and Success. <code>domain/handover.ts</code> ; <code>application/handover.ts</code> ; <code>adapters/postgres/store.ts</code> .
<code>workforce/</code>	Availability, teams, shifts, dated assignments and revisions. <code>domain/workforce.ts</code> ; <code>application/workforce.ts</code> ; <code>adapters/postgres/store.ts</code> and <code>assignment-default.ts</code> .
<code>maintenance/</code>	Work lifecycle, permissions, priorities, reviewed repair decisions and completion. <code>domain/maintenance.ts</code> ; <code>application/maintenance.ts</code> ; <code>adapters/postgres/store.ts</code> .
<code>assets/</code>	Stable asset identities, aliases, lifecycle and source-authorized timeline. <code>domain/assets.ts</code> ; <code>application/assets.ts</code> ; <code>adapters/postgres/store.ts</code> .

The entry point for wiring these owners is `apps/api/src/host/runtime.ts`. `host/*-controller.ts` translates transport calls; `host/*-contracts.ts` defines DTOs. `apps/api/contracts/openapi.json` is the reviewed generated HTTP contract.

Persistence: owning stores enforce their own writes and revision rules. `site-operation.ts` pins a connection, checks current grants and installs transaction-local organization/site/actor context. Forced row-level security is additional isolation, not a substitute for business permission checks. The API uses `iop_runtime`, not an owner.

Cross-module example: Success is validated by Handover. Maintenance completes selected work through its own receiving contract on the same transaction, retaining repair review. Asset timelines call source-owned readers; asset access does not grant access to Handover, Maintenance or Analysis data.

04 / Frontend layout and component map

`apps/web/src/host/AnalyticalApp.tsx` constructs application objects and HTTP gateways. Despite its historical name, it composes the whole operational application. `WorkspaceApp.tsx` owns session context, profile presentation, navigation and page selection.

Shared shell: desktop sidebar, top account/notification controls, breadcrumb/page heading, content surfaces and active workspace controls. Narrow layouts reuse these controls. Administrator Start composes administrative summaries; operational Start composes personal assignments and authorized report/work summaries. Active Information appears on every profile's Start.

Feature below <code>apps/web/src/features/</code>	Visible layout and main components
<code>analysis/</code>	<code>StartOverview</code> and <code>Workspace</code> : report selector, collapsible filters, charts/KPIs. <code>ImportWorkspace</code> , <code>SourceFiles</code> and <code>ProfileEditor</code> handle administration. <code>adapters/echarts/charts.ts</code> maps server measures into plots.
<code>access/</code>	<code>LoginPanel</code> ; <code>UserAdministration</code> with create/details/profile dialogs. Account/profile preview is shell presentation, not a server identity change.
<code>shift-handover/</code>	<code>HandoverWorkspace</code> : Meeting preparation/Daily overview, personal Journal, Department matrix. <code>MeetingCanvas</code> contains category panels and pending disclosure. <code>EntryForm</code> , <code>EntryDetail</code> and <code>SuccessReferences</code> own the form/detail views; <code>InformationNotices</code> supplies Start notices.
<code>workforce/</code>	<code>WorkforceWorkspace</code> : dated/weekly planning, settings and imports. <code>PlanBoard/DailyPlan</code> show assignments; <code>WeeklySchedule</code> , <code>AssignmentForm</code> and <code>RecordDetails</code> handle planning and history.
<code>maintenance/</code>	<code>MaintenanceWorkspace</code> with board/search, filters and work details. <code>MaintenanceForm</code> handles work; <code>RepairScope</code> handles explicit report inclusion/exclusion; <code>MaintenanceStatusDialog</code> handles focused status changes.
<code>assets/</code>	<code>AssetsWorkspace</code> : registry/search and selected asset. <code>AssetForm</code> , <code>AssetDetails</code> , <code>AssetAliasFields</code> and <code>AssetTimeline</code> handle identity and authorized source history.

Inside each feature: `domain/` contains browser values/models; `application/` coordinates framework-free use cases; `adapters/http/gateway.ts` calls the API; `adapters/react/` contains components and feature CSS. Read models are not server persistence entities.

Shared file	Purpose
<code>apps/web/src/design/components/</code>	Shared controls; <code>index.ts</code> exports buttons, fields, dialogs, panels and navigation.
<code>apps/web/src/design/identity.ts</code> and <code>base.css</code>	Canonical visual identity and base styles.
<code>apps/web/src/localization/i18n.ts</code> and <code>de.ts</code>	English source/default strings and German resources.
<code>apps/web/src/contracts/schema.d.ts</code>	Generated OpenAPI bindings; regenerate rather than edit.
<code>apps/web/vite.config.ts</code>	Loopback ports, React plugin and proxy to API port 3000.

Use shared controls and tokens. Feature CSS arranges a view; it must not invent a new font/palette. Read `docs/design/visual-identity.md` before UI changes.

05 / Prepare once, before the trip

This recipe assumes the same Mac that already holds `.local-platform/` and the database volume. A Git clone on another laptop does not contain the database or credentials. Moving to another machine needs a separately verified database transfer.

Open Terminal in the repository root:

```
cd ~/Desktop/VictorTilve/Switzerland/Galaxus/Techniker/IOP/iop-platform
nvm install
nvm use
node --version
npm --version
npm ci
```

Command	What it does
cd	Sets the root for every relative path in this guide. If the folder moved, use its actual path.
nvm install / nvm use	Installs/selects the <code>.nvmrc</code> Node version, currently 24.21.0. Run <code>nvm use</code> in each terminal.
node --version / npm --version	Confirm Node 24.21.0 and npm 10.9.2. The project requires Node 24, not Node 20.
npm ci	Installs exactly <code>package-lock.json</code> into the workspaces. It needs network on first preparation and replaces <code>node_modules</code> .

If `nvm` is installed but unavailable in a new terminal, load its shell initialization:

```
export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && . "$NVM_DIR/nvm.sh"
nvm use
```

If `npm` differs from the documented version, prepare it while online:

```
npm install --global npm@10.9.2
```

Keep locally: the repository and `node_modules`, Docker Desktop, the existing `postgres:17.6-bookworm` image and persistent volume, and the complete private `.local-platform/` installation. Preserve `scope.json`, `users.json`, `mappings.json`, `handover.json`, `runtime.env` and `setup.env`. Do not commit or place credentials in Vite.

What remains unchanged: organization/site/source IDs, source mappings, locations, existing password hashes/accounts and business data. The preparation on the next page copies browser-origin settings and reuses the runtime credential; it does not seed users. Never use `demo:reset/demo:recreate`, volume deletion or fixture `--apply` as startup steps.

06 / Prepare private native settings

Run this once from the root. Re-run if the repository moves or the runtime credential or original identity configuration changes. It writes only ignored private files. The added origin is the Vite browser address, not a permission grant.

```
node <<'NODE'
const fs = require('node:fs');
const path = require('node:path');
const dir = path.resolve('.local-platform/native');
fs.mkdirSync(dir, {recursive: true, mode: 0o700});
const users = JSON.parse(
  fs.readFileSync('.local-platform/config/users.json', 'utf8'));
users.origins = [...new Set([...users.origins,
  'http://127.0.0.1:5173'])];
if (users.origins.length > 4) throw Error('Review origin capacity.');
```

```
fs.writeFileSync(path.join(dir, 'users.json'),
  JSON.stringify(users, null, 2), {mode: 0o600});
const secret = fs.readFileSync('.local-platform/runtime.env', 'utf8')
  .split(/\r?\n/).find(x => x.startsWith('IOP_RUNTIME_PASSWORD='));
if (!secret) throw Error('Existing runtime credential is missing.');
```

```
const files = {
  IOP_CONFIG_FILE: '.local-platform/config/scope.json',
  IOP_LOCAL_IDENTITY_FILE: '.local-platform/native/users.json',
  IOP_MAPPING_CONFIG_FILE: '.local-platform/config/mappings.json',
  IOP_HANDOVER_CONFIG_FILE: '.local-platform/config/handover.json'
};
const values = [
  'NODE_ENV=development', 'IOP_EXECUTION_MODE=local-demo',
  'IOP_AUTHENTICATION=password', 'IOP_TRANSPORT=native',
  'HOST=127.0.0.1', 'PORT=3000', 'IOP_DATABASE_MODE=local',
  'IOP_DATABASE_HOST=127.0.0.1', 'IOP_DATABASE_PORT=5442',
  'IOP_DATABASE_NAME=iop_local', secret,
  ...Object.entries(files).map(([key, file]) =>
    key + '=' + JSON.stringify(path.resolve(file)))
];
fs.writeFileSync(path.join(dir, 'runtime.env'),
  values.join('\n') + '\n', {mode: 0o600});
fs.writeFileSync(path.join(dir, 'compose.yaml'),
  'services:\n  database:\n    ports:\n' +
  '    - "127.0.0.1:5442:5432"\n', {mode: 0o600});
console.log('Private native settings prepared.');
```

NODE

runtime.env contains the current runtime password plus explicit native settings. Node loads it only when --env-file is supplied. JSON file paths are absolute. The private users.json preserves configured user IDs and adds the allowed Vite origin. The private Compose overlay publishes container port 5432 on host loopback port 5442. It keeps the same Compose project and volume; no new database is created.

The mode name local-demo is a historical execution label. With IOP_AUTHENTICATION=password it uses existing local login, not demo-user impersonation. Existing exported environment variables override env-file values; use a clean terminal if you previously exported incompatible IOP settings.

07 / Daily startup: database plus three terminals

Start Docker Desktop. From the root, stop the container applications before exposing the database. The single-host lease rejects a second API on this database.

```
docker compose -f compose.platform.yaml stop web api
docker compose -f compose.platform.yaml \
  -f .local-platform/native/compose.yaml up -d --wait database
npm run build --workspace @iop/api
```

stop web api leaves PostgreSQL and its volume intact. The overlay may recreate the database container to add the port; its persistent volume is reused. --wait checks container health. The API build creates current JavaScript once; it cleans API dist/. Do not pass a backup argument or run provisioning/seeding for this existing database.

Terminal A - compiler. Select Node with nvm use, then leave this running:

```
node node_modules/typescript/bin/tsc \
  -p apps/api/tsconfig.build.json --watch --preserveWatchOutput
```

TypeScript recompiles changed API source into apps/api/dist/. Wait for zero errors.

Terminal B - API. From the root with nvm use:

```
node --env-file=.local-platform/native/runtime.env \
  --watch --watch-preserve-output apps/api/dist/main.js
```

Node runs the compiled API and restarts when loaded JavaScript changes. The browser sees current backend behavior on its next request. Reload the page or repeat the action after an API restart; backend watch does not itself refresh React.

Terminal C - frontend. From the root with nvm use:

```
npm run dev:web
```

Vite serves source and sends React/CSS updates to the browser. Open <http://127.0.0.1:5173> and log in with an existing account. Use 127.0.0.1 exactly, not localhost; the origin allowlist is exact. /api and /health proxy to port 3000.

Verify without changing data:

```
curl --fail http://127.0.0.1:5173/health
curl --fail http://127.0.0.1:5173/api/v1/session/context
```

Health should return status ok. Context should be enabled with password authentication; user null is normal before login. An enabled context is more informative than health.

08 / Follow a change through the code

Frontend edits to React/CSS use Vite hot updates. Changes to application state or composition may cause a full refresh or require reopening the screen. Unsaved forms can be lost on refresh. Existing API mutations write to the real retained database.

Backend edits follow source -> TypeScript output -> Node restart -> next browser request. Do not edit dist/ to implement a feature. If TypeScript reports an error, fix it before evaluating the browser. Use the three terminal logs and browser DevTools.

Example: a new Handover entry

1. EntryForm.tsx renders controls and collects the draft. application/entry-draft.ts handles category/condition draft transitions.
2. The browser application/handover.ts coordinates the use case; adapters/http/gateway.ts sends a same-origin /api/v1/handover request.
3. API host/handover-controller.ts receives HTTP input; handover-contracts.ts owns transport DTOs. Host/runtime.ts supplies the configured catalog and actor.
4. API modules/shift-handover/application/handover.ts orchestrates current permissions, revision checks and writes. domain/handover.ts validates the content and classification.
5. adapters/postgres/store.ts uses persistence/site-operation.ts. Handover stores the current projection and immutable revisions on one authorized connection.
6. The response updates browser state and the view. Success may additionally invoke the Maintenance-owned completion contract within the same transaction.

Paths for steps 1-2 begin apps/web/src/features/shift-handover/. Paths for steps 3-5 begin apps/api/src/.

First reading exercise: inspect EntryForm.tsx, the HTTP gateway, handover-controller.ts and Handover.create, then follow validContent into the domain. Compare apps/api/test/handover-workflows.spec.ts with the implementation. Observe that Safety validation exists on the server as well as in the form.

Small UI exercise: locate a source/default label or layout in the owning React component; change it on your planned story branch and observe the hot update. Use shared design tokens. Add German text only in localization resources.

Small API exercise: change one authorized validation rule on a planned story, update a behavior test, watch compilation/restart, then repeat the browser request. Do not weaken permissions or RLS to simplify debugging.

In DevTools Network inspect URL, request fields, response status and Problem Details. A 400/403/409 usually reflects validation/permission/revision handling; a 5xx traceId correlates the sanitized server error. Generated bindings must match the API artifact.

09 / Checks, contracts and schema commands

Stop compiler/API watchers before commands that rebuild dist/. Root npm test, build, openapi, database tests and browser tests rebuild output; running them beside the watcher can trigger restarts or missing-module/port conflicts. Ctrl-C stops a terminal.

Root command	What it does
npm run typecheck	Checks API, web and database tooling TypeScript without emitting output.
npm test	Runs secret-tooling tests, builds, design/contract checks, API/web unit tests and database configuration tests. It does not test the live operator database.
npm test --workspace @iop/api	Runs API tests only; includes HTTP listener tests.
npm test --workspace @iop/web	Runs browser application/component tests without a real browser.
npm run test:database	Builds and runs isolated real PostgreSQL integration tests. Requires Docker; disposable Testcontainers do not use the operator volume.
npm run test:e2e	Builds and runs Chromium against temporary API/web servers. Stop native API first: the suite needs port 3000 and preview port 4173.
npm run check:design --workspace @iop/web	Checks shared visual-identity/boundary rules.
npm run check:secrets	Inspects indexed files for bounded secret/private-file patterns; manual review is still needed.

After an authorized HTTP contract change:

```
npm run openapi
npm run contract --workspace @iop/web
npm run contract:check --workspace @iop/web
```

openapi builds and exports Nest metadata into apps/api/contracts/openapi.json. contract regenerates apps/web/src/contracts/schema.d.ts. contract:check rejects drift. Review both generated diffs. There is no served Swagger UI.

Schema work only when your planned story adds reviewed migrations:

```
npm run db:build
node --env-file=.local-platform/native/runtime.env \
  --env-file=.local-platform/setup.env \
  infra/database/dist/cli.js migrate
```

db:build compiles the database CLI. migrate uses iop_migrator and ordered SQL/history; it is a real change to the existing database. The second private env file supplies the migration credential to this one-shot tool, not the running API. IOP-199 requires no new migration. Do not run db:provision or seed commands merely to start native apps. Applied migrations stay immutable; append a new reviewed file rather than editing history.

10 / Troubleshooting and returning to Docker

Symptom	Check and response
Only health works / context disabled	Business activation is missing. Use the explicit native env file and current scope/mapping/identity files. Health alone does not prove access to data.
Another application host is active	Stop Docker api and any prior native API. One database permits one application host. Do not remove the lease.
Connection refused / startup failure	Start Docker Desktop; run the private database overlay; verify the 5442 loopback mapping and existing runtime credential. Do not generate new passwords.
403 from browser	Use http://127.0.0.1:5173 exactly; ensure that origin is in native/users.json; restart API after config changes. Current server grants can also deny the action.
Login required / expired session	Log in with an existing account. Session expiry or password/profile changes can invalidate access. View as does not change the server actor.
EADDRINUSE or Vite strict-port failure	Another process owns 3000/5173. Stop it; changing ports also requires proxy/origin settings.
Backend change seems invisible	Check compiler errors and Node restart log; repeat the request or refresh the page. Check that the browser is on 5173 rather than Docker 8080.
Config/mapping/location change invisible	Restart Node explicitly; startup reads JSON configuration once. Native users.json is a separate copy of the origin configuration.
Maintenance Success reports conflict	Review current work revision and complete include/exclude repair scope. A failed Success closes nothing.
Administrator cannot publish Information	The default publisher profile is Team Leader, with current coordinator grants. Profile preview does not confer that identity.

End native development: press Ctrl-C in terminals A, B and C. The database may remain running for your next session. To stop it without deleting its volume:

```
docker compose -f compose.platform.yaml \
  -f .local-platform/native/compose.yaml stop database
```

Return to the standard container application: after all native watchers stop:

```
npm run local:up
npm run local:status
```

local:up preserves credentials/data, rebuilds web/API/setup, applies pending migrations and verifies retained analytics. It restores the standard database service definition without the native port overlay. local:status prints service state. Open http://127.0.0.1:8080. Keep the private native files for the next development session. Do not use compose down --volumes: that removes the retained database.

11 / Verified state and remaining work

As of 2026-10-09, IOP-199 implementation validation passed:

Check	Evidence
npm test	418 API tests, 149 web tests, 77 database configuration tests and 18 secret-tooling tests; builds, design guard and contract consistency passed.
Typecheck	Both applications and database tooling passed.
Scoped PostgreSQL suites	42 tests, including atomic selected closure, conflicts and rollback after Maintenance writes.
Selected real-browser scenarios	12 scenarios across desktop/tablet/narrow layouts; rendered Information, Safety, Success and personal Journal inspected.
Docker activation	API/web/database healthy, enabled password context, IOP-199 code present and retained counts unchanged.

Retained record counts during activation: Handover 67; Maintenance 34; Workforce 1,606; Assets 5,655; import attempts 104. These include synthetic exercises and do not measure physical equipment health. Seed verification retained 78 dates and 42,220 source rows; no seed dates were imported during this update.

Remaining scope comes from the canonical backlog, not this snapshot's test count:

Area	Remaining outcome
Identity/shared operation	Corporate/provider login and broader multi-organization/site lifecycle.
Assets	Full composition/types/controller links, physical survey and ingestion-time canonical mapping.
Formal Handover	Shift-bound records, formal closure/acknowledgment and complete canonical references. Dated Workforce defaults are already delivered in IOP-199.
Integrations/analysis	External read-only WinCC/Ultimo/Entra contracts and broader canonical Pareto coverage.
Improvement Tracking	Inclusion/ownership decision and action/evidence/impact workflow.
Infrastructure/operations	CI, worker/retry infrastructure, general Audit, verified full backup/restore and performance targets.
UX/release	Unified area/global search, consolidated pilot measurements and shared-use/release controls.
Asset Locator / M9	Maps and placement are explicitly Deferred.

Data Analysis charts were accepted as v1; IOP-130 remains open for measurement consolidation. Local implementation, Git publication and whole-platform release are different states. Stage/master promotion and shared deployment were not performed. The optional legacy IOP-173 demo writer needs adaptation to the new category rules; do not prepare/apply its old fixtures on this workflow.

12 / Offline routine, review and sources

Before leaving: run the native startup once while online, install dependencies, open the required screens and verify login. Cache the Docker database image and Playwright browser if you want E2E tests. Keep this PDF locally; keep private settings and the existing volume on the same Mac. Git/network dependency downloads are not available offline, while the prepared native apps plus local database can run.

```
npx --no-install playwright install chromium
git status --short --branch
```

The first command downloads the browser when needed, so do it before travel. git status distinguishes your branch, tracked edits and untracked files. Do not assume copying source alone carries database state.

Start a future story after selecting its scope: inspect backlog/item/accepted ADRs, create its execution plan, then branch from develop. These are commands you run for your own review workflow; the guide does not authorize automatic merges/publication.

```
git switch develop
git pull --ff-only origin develop
git switch -c feature/IOP-NNN-short-title
git diff
git diff --check
```

Use a real allocated story ID/name, and commit only that story. The fast-forward-only pull refuses divergent history. While offline, use your already synchronized develop. Run relevant checks; then create logical English commits. Preserve other review branches. Promotion remains story -> develop -> stage -> master with explicit owner authorization.

Canonical reading order / source index

Source in the repository	Use
README.md, ARCHITECTURE.md	Entry point, architecture and source ownership.
docs/architecture/modules.md and data-model.md	Logical modules, contracts and persisted concepts.
docs/architecture/adr/	Read Accepted decisions relevant to your change; Proposed is not Accepted.
docs/planning/backlog.md and poc-delivery.md	Current story statuses and delivered evidence; older review dates are historical snapshots.
ROADMAP.md	Consolidated remaining work; IOP-199 supersedes its earlier Workforce-default deferral.
docs/product/shift-handover.md	Current operational category/Journal/Success contract.
docs/development/running-poc.md	Standard Docker operator workflow and seed provenance.
docs/development/local-configuration.md	Scope, origins and explicit environment/configuration rules.
infra/database/README.md	Roles, migrations, RLS and native database-tool contracts.
docs/development/testing-poc.md	Test layers and prerequisites.
docs/design/visual-identity.md	Shared UI roles, tokens and rendering checks.
docs/planning/workflow.md	Planning, English content, logical commits and publication convention.